

Neural processes

Learn from many related functions how to turn a few observed points into a prediction for a new one: a hypernetwork outputs weights, a neural process encodes, averages and decodes.

THE QUESTION

Given a handful of (x, y) points from a function we have never seen, can a network predict the rest of it without being trained on that function?

THE INTUITION

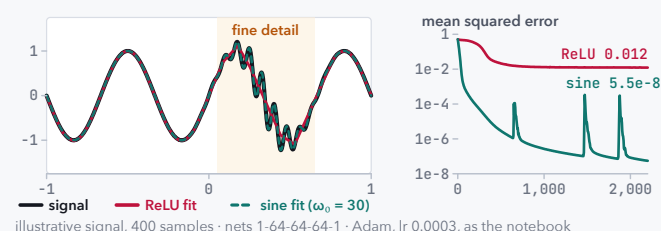
Train on many related tasks instead of one. Split each task into context and target points and learn the map from the context to predictions at the targets. A neural process encodes each context pair, averages the codes and decodes at any query x_* , so the order of the points does not matter.

KEY TAKEAWAY

$(\mu_*, \sigma_*) = g\left(\frac{1}{n} \sum_i h(x_i, y_i), x_*\right)$.
Trained on a Gaussian likelihood it is a conditional NP, a learned stand-in for GP prediction. Trained on squared error it returns one curve and no uncertainty.

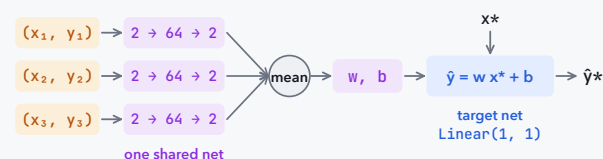
many tasks $f_1, f_2, \dots \rightarrow$ context $(x_i, y_i) \rightarrow$ encode $h(x_i, y_i) \rightarrow$ average $r \rightarrow$ decode $g(r, x^*) \rightarrow \mu(x^*), \sigma(x^*)$

1 A coordinate network stores one function



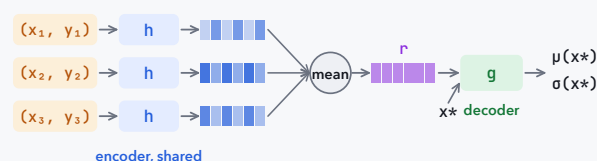
A coordinate net maps a location to a value, $x \mapsto f(x)$ or $(row, col) \mapsto (R, G, B)$: its weights are the signal. SIREN uses $h_{l+1} = \sin(\omega_0(W_l h_l + b_l))$ with $\omega_0 = 30$, a first layer $U(-\frac{1}{n}, \frac{1}{n})$ and later ones $U(\pm\sqrt{6/n}/\omega_0)$. ReLU nets favour smooth functions and miss fine detail. The notebook fits a 400×400 photo crop (160 000 pixels) with 4 hidden layers of 128, 50 307 weights; its comment says 5 layers of 256 (264 707). Quoted, after 2200 Adam steps: MSE 0.0091 with ReLU, 0.00032 with sine.

2 A hypernetwork turns the context into weights



A fresh net per function needs many points: sine nets trained on 5, 10, 100, 1000 and 10 000 pixels score 0.23, 0.043, 0.029, 0.014 and 0.0040 on the whole crop (quoted). Meta-learning trains across many tasks of one family instead, here lines $y = \alpha + \beta x + \epsilon$. The notebook's hypernetwork maps each context pair through one SELU net and averages the outputs into the weight and bias of `Linear(1, 1)`. Averaged votes cannot form the least-squares slope $\sum x_i y_i / \sum x_i^2$, a ratio of means: at 10 points its MSE is 0.94 against 0.11. Emitting all 50 307 coordinate-net weights needs a $512 \rightarrow 50\,307$ layer, 25.8 million weights.

3 A neural process encodes, averages, then decodes

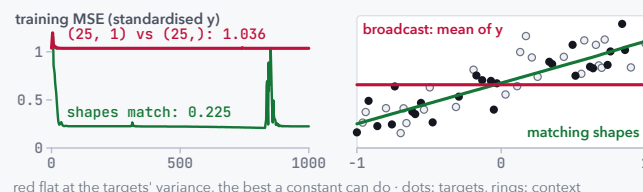


order of the context does not matter (a mean) · any number of points · any query x^*

$$r = \frac{1}{n} \sum_{i=1}^n h(x_i, y_i), \quad (\mu_*, \sigma_*) = g(r, x_*)$$

h and g are small MLPs shared by all tasks. The mean ignores the order of the context and allows any n ; g can combine mean features, the ratio above included. Training: sample a task, 1 to 10 context points and 50 targets; minimise $\log \sigma_* + (y_* - \mu_*)^2 / (2\sigma_*^2)$ over the targets. This is the conditional NP (CNP). The notebook's NP (encoder 2-8-8, decoder 9-8-1) uses squared error and outputs μ_* only. Cost: $O(n + m)$.

4 The notebook's NP never learns: a shape mismatch



```
y_pred = decoder(joint) # (25, 1)
l = criterion(y_pred, y_t) # (25,)
# broadcasts to (25, 25); the fix:
l = criterion(y_pred.squeeze(-1), y_t)
```

Every prediction p_i is scored against every target y_j :

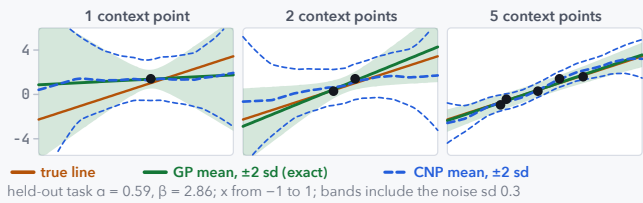
$$\frac{1}{n^2} \sum_{i,j} (p_i - y_j)^2 = \frac{1}{n} \sum_i (p_i - \bar{y})^2 + \text{Var}(y).$$

The best output is the constant $\bar{y}$; the floor is the targets' variance. The notebook's losses stay at 1.00 and 0.72 (quoted), under a torch size warning. Rerun on one standardised line task: 1.036, the targets' variance, against 0.225 with matching shapes (noise alone: 0.26).

From one curve to a predictive distribution

A conditional NP imitates GP prediction one x at a time. A net's uncertainty has a noise part and a weight part.

5 A conditional NP learns to imitate GP prediction

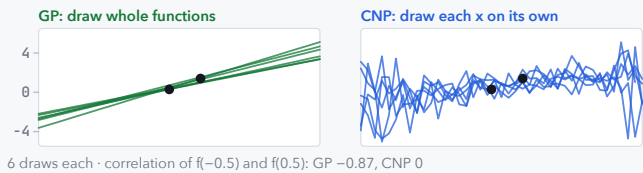


Lines with $\alpha \sim \mathcal{N}(0, 2^2)$, $\beta \sim \mathcal{N}(0, 3^2)$ form a GP with kernel $k(x, x') = 4 + 9xx'$. With noise sd 0.3 its predictive is exact: mean $x_*^T m_N$, variance $x_*^T S_N x_* + \sigma^2$ (the Bayesian linear regression sheet). The CNP sees only 2000 sampled tasks. Test MSE on 300 new tasks (noise alone 0.09, predicting 0 gives 7.15):

context points	1	2	3	5	10
GP (exact)	3.24	1.08	0.30	0.15	0.11
CNP	3.43	1.55	0.68	0.24	0.14
hypernetwork	4.49	2.71	1.81	1.28	0.94
least squares	none	8.87	0.34	0.15	0.11

Test NLL at 2 points: GP 0.98, CNP 1.18; the CNP's $\pm 2\sigma_*$ holds 94 to 96% of targets. Least squares at 2 points: median 0.31, mean 8.87 from near-vertical fits. A GP needs the right kernel and $O(n^3)$ algebra; an NP needs many tasks, then predicts in one pass.

6 CNP marginals are not a distribution over functions

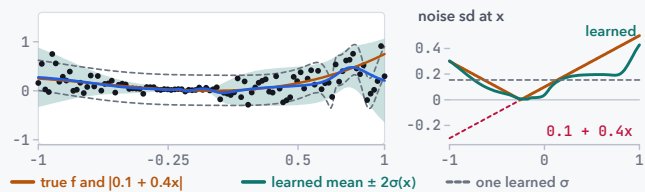


A CNP gives $p(y_* | x_*, C)$ one query at a time; draws at different x_* are independent, so they are not functions. A latent NP adds a global $z \sim q(z | C)$ from the averaged code and decodes $g(z, x_*)$: one z , one coherent function, trained with an ELBO (the variational inference sheet). The notebook's NP is deterministic and fit on squared error. "A distribution over functions" describes the latent NP and the GP, not that model.

10 Pause and think

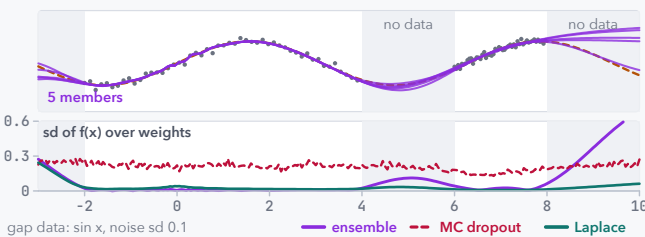
Why average the codes instead of concatenating them? The code keeps one size for any n , and the order of the points cannot matter. Which part does more data shrink? The epistemic one. $\sigma^2(x)$ belongs to the data.

7 Aleatoric: let the net predict the noise



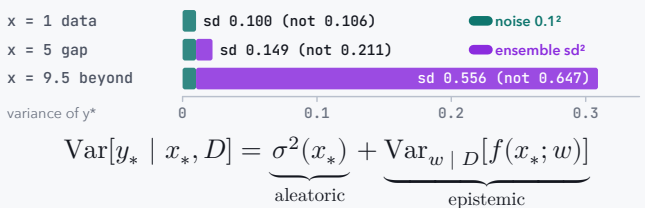
Noise in y at a fixed x stays however much data we add. Output $\mu(x)$ and $\log \sigma(x)$; minimise $\frac{1}{N} \sum_i [\log \sigma(x_i) + (y_i - \mu(x_i))^2 / (2\sigma(x_i)^2)]$. The notebook tries σ fixed at 0.5, one learned σ , and $\sigma(x)$. Its heteroskedastic noise is $\mathcal{N}(0, 1) \cdot (0.1 + 0.4x)$; the factor is negative for $x < -0.25$, so the sd is $|0.1 + 0.4x|$. Rerun: one $\sigma = 0.154$ (NLL -0.45); $\sigma(x) = 0.30, 0.007, 0.43$ at $x = -1, -0.25, 1$ (NLL -0.87 ; notebook, quoted: $-0.063, -0.602$).

8 Epistemic: how far plausible weights disagree



Doubt about w shrinks with data. Gap data, 1-32-32-1 tanh nets, $\mathcal{N}(0, 1)$ initial weights as in the notebook: 5 ensemble members, MC dropout ($p = 0.1$, 100 passes), last-layer Laplace ($\Sigma = (\Phi^T \Phi / \sigma^2 + I)^{-1}$ on 33 output weights). The sd at $x = 1, 5, 9.5$: ensemble 0.006, 0.11, 0.55; dropout 0.24, 0.19, 0.22; Laplace 0.020, 0.033, 0.049. Only the ensemble grows off the data, and at 9.5 four members sit at 1.0 to 1.5 while $\sin 9.5 = -0.08$.

9 Add the two parts as variances



The notebook's Laplace cell adds sds, $0.2 + \sigma_w$ (0.3 for $\sigma_w = 0.1$, not 0.224), and assumes noise 0.2 on the heteroskedastic data: ± 0.4 around the true f holds all points on $-0.5 < x < 0$ but 76% for $x > 0.5$.

encode → average → decode · train across tasks · predict $\sigma(x^*)$ too · variance = $\sigma^2 + \text{Var over weights}$

Source: slides "Neural processes" (one frame, repeating the Bayesian optimisation stub), the notebooks on hypernetworks and neural processes, coordinate networks, uncertainty in neural nets and MC dropout, and the Bayesian linear regression slides, nipunbatra.github.io/pml-teaching. Numbers recomputed in NumPy (MLPs with hand-written gradients, fixed seeds); torch and GPU outputs are quoted. Line tasks here use noise 0.3; the notebook draws it from HalfNormal(5) and standardises each task. Fixes: the NP loss broadcasts (25, 1) against (25,); the SIREN comment says 5 layers of 256; its "test" error uses the training pixels; "predict on new task" reuses a training task; the uncertainty notebook adds sds, treats $0.1 + 0.4x$ as an sd, and fits Laplace with noise 0.2 on heteroskedastic data.